Assignment 2: Mixture of Experts, Optimizers and Decoding Strategies

Likhit Bhogadi · bhogadi.likhit@students.iiit.ac.in

Q1 WandB: https://wandb.ai/likhitbhogadi-iiit-hyderabad/anlp-a2-q1 · HF checkpoints: `huggingface.co/likhitbhogadi/anlp-a2-q1-<variant>` for dense, moe_4e_top1, moe_4e_top2, moe_shared1_3e_top1, moe_4e_top2_active
Q2 WandB: https://wandb.ai/likhitbhogadi-iiit-hyderabad/anlp-a2-q2 · HF checkpoints: `huggingface.co/likhitbhogadi/anlp-a2-q2-<optimizer>` for adamw, mars, lion, muon
Q3 WandB: https://wandb.ai/likhitbhogadi-iiit-hyderabad/anlp-a2-q3 · raw generations: https://huggingface.co/datasets/likhitbhogadi/anlp-a2-q3-generations
Code: `src/part1`, `src/part2`, `src/part3`. Longer per-question write-ups: `src/part1/report.md`, `src/part2/report.md`.

1. Mixture of Experts

1.1 Setup

Data. We translate Vietnamese→English and Japanese→English with `belumind/en-vi-ja-curated-500k-triplets` (446,252 / 24,792 / 24,792 triplets); each triplet gives a vi→en and a ja→en pair. A pair is one decoder-only sequence, `<bos> <vi|ja> source <2en> target <eos>`, with the loss on target tokens and `<eos>` only. **Tokenizer:** one shared 32k BPE, trained on the train split without romanisation (NFKC, Metaspace pre-tokenisation). Japanese still forms multi-character merges (クエスト, ません。), and byte fallback removes `<unk>`. **Equal token budget:** after dropping pairs over 160 tokens (0.57% of triplets), every model sees exactly **30,000,153 training tokens** in the same order: the shortest prefix of one fixed shuffle that reaches 30M (388,936 triplets, one epoch of 12,155 steps × 64). All 49,314 usable test pairs are used for perplexity and BLEU.

Model. A pre-norm decoder with RMSNorm, RoPE and causal SDPA attention, d_{model} 512, 6 layers, 8 heads and tied embeddings. Only the FFN changes (Table 1). Variants 2–4 split dense's 2,048 hidden units into experts of 512, so *total* parameters match. Variant 5 doubles the expert width, so 2 active experts × 1,024 match dense's *active* parameters.

Table 1. The five FFN variants. Active parameters count everything a token uses: embeddings, attention, router, shared expert and its k routed experts.

#	variant	FFN in every layer	k	total	active
1	`dense`	GELU MLP 512→2048→512	–	35.27M	35.27M
2	`moe_4e_top1`	4 experts × (512→512→512)	1	35.28M	25.84M
3	`moe_4e_top2`	4 experts × (512→512→512)	2	35.28M	28.99M
4	`moe_shared1_3e_top1`	1 always-on shared + 3 routed experts (512→512→512)	1	35.27M	28.98M
5	`moe_4e_top2_active`	4 experts × (512→1024→512)	2	47.86M	35.28M

MoE layer (`model.MoE`, from scratch). A linear router and softmax pick the top-k experts. For k = 2 the two gates are renormalised; for k = 1 the raw probability is kept, since a renormalised gate is always 1 and the router would get no LM-loss gradient. There is no capacity factor, so no token is dropped: each expert gathers its tokens (`index_select`), runs its MLP and scatter-adds the gated outputs (`index_add_`). The shared expert is added ungated, and padding is never routed. Auxiliary losses are the Switch load-balancing loss $E \cdot \Sigma_i f_i P_i$ (weight 0.01; 1.0 means uniform load) and a router z-loss (weight 0.001). A self-test checks the dispatch against a per-token loop, padding invariance, and KV-cached decoding against a full forward pass.

Training. AdamW with β = (0.9, 0.95) and weight decay 0.1 on matrices; 500 warmup steps, then cosine decay to 10%; clipping at 1.0; bf16; batch 64; one seed. A full-budget LR sweep chose **6e-4** for all runs (final val CE at 3e-4 / 6e-4 / 1.2e-3: dense 2.534 / **2.413** / 2.562, `moe_4e_top1` 2.618 / **2.576** / 2.578).

Evaluation. Perplexity is exp(mean target cross-entropy), teacher-forced. BLEU is corpus sacreBLEU (13a) from our own batched greedy decoder (left padding with RoPE position correction, KV cache, at most 96 new tokens). During the test pass we count routing assignments per layer and expert, split by source language and by token side: *source* (`<bos> <lang>` + source) versus *target side* (`<2en>` + generated English).

1.2 Results

Table 2. Test results on all 49,314 pairs.

variant	total	active	PPL vi	PPL ja	PPL	BLEU vi	BLEU ja	BLEU
dense	35.27M	35.27M	8.26	**13.36**	**10.51**	34.01	24.11	29.09
moe_4e_top1	35.28M	25.84M	9.55	15.98	12.35	31.62	21.87	26.77
moe_4e_top2	35.28M	28.99M	8.75	14.48	11.26	32.93	22.73	27.79
moe_shared1_3e_top1	35.27M	28.98M	8.64	14.07	11.02	32.99	23.07	28.05
moe_4e_top2_active	47.86M	35.28M	**8.23**	13.45	10.52	**34.37**	**24.30**	**29.36**

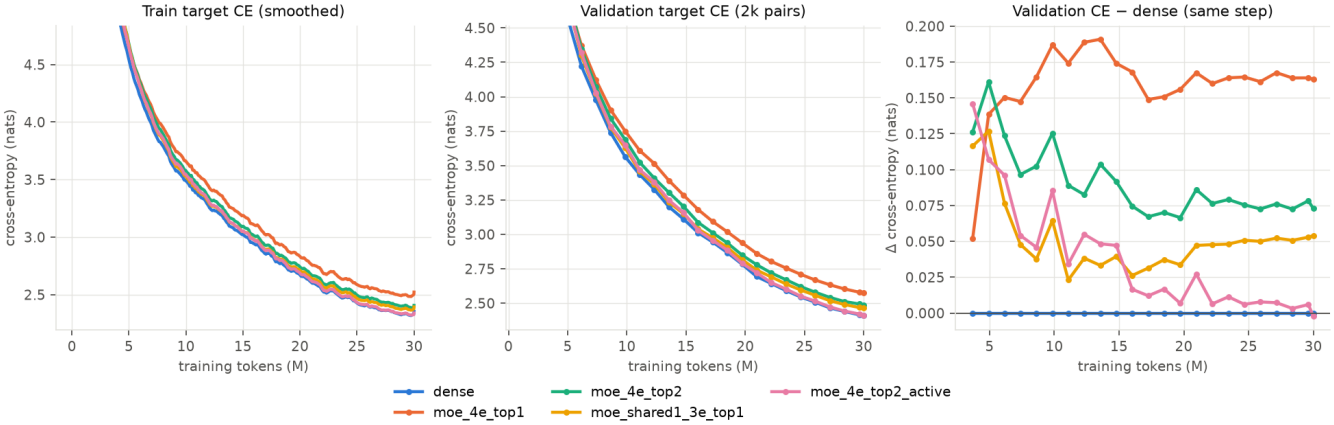


Figure 1. Train and validation target cross-entropy against tokens seen, and each model's validation gap to dense at the same step.

At equal total parameters (1–4), quality follows active parameters. Every MoE with dense's parameter count is worse than dense, in order of active FFN size: dense (12.6M active FFN parameters) PPL 10.51; top-2 and shared+top-1 (6.3M) 11.26 and 11.02; top-1 (3.2M) 12.35. BLEU falls in the same order, from 29.1 to 26.8. Splitting a fixed budget into experts doesn't help here, because each token then uses only part of the FFN. The gaps to dense settle after about 15M tokens and are not closing (Fig. 1, right): +0.16 nats for top-1, +0.075 for top-2 and +0.05 for shared+top-1.

At equal active parameters, the larger MoE matches dense. `moe_4e_top2_active` (twice dense's FFN parameters, same compute per token) ties dense on perplexity (10.52 vs 10.51) and is slightly better on BLEU (+0.27; +0.36 vi→en, +0.19 ja→en). Its validation gap shrinks steadily from +0.15 nats to about 0, and goes slightly negative at the last evaluation. Each expert sees only about half the tokens, so at 30M tokens the experts are under-trained. A longer budget would probably widen the lead, but here the BLEU difference is within noise.

A shared expert beats plain top-2 at the same active size (PPL 11.02 vs 11.26, BLEU 28.05 vs 27.79, for both languages). The always-on expert learns what every token needs, which frees the routed experts to specialise, and choosing 1 of 3 is an easier routing problem than 2 of 4. **Top-2 beats top-1** (11.26 vs 12.35), partly through twice the active size and partly through smoother gradients from mixing two experts.

Japanese is harder for every variant: PPL 13.4–16.0 against 8.2–9.6, and about 10 BLEU lower. Japanese is SOV and unspaced and needs long-range reordering; Vietnamese is Latin-script SVO. The variant ranking is the same for both languages. **Cost:** dense trains at about 57k tokens/s and the MoEs at about 35k on one L40S, because of our simple per-expert dispatch loop, not extra FLOPs. **Samples:** all models' outputs are near-identical on easy sentences, and every model's main failure is greedy repetition on long inputs (… `of the year of the year` …).

1.3 Expert usage and specialisation

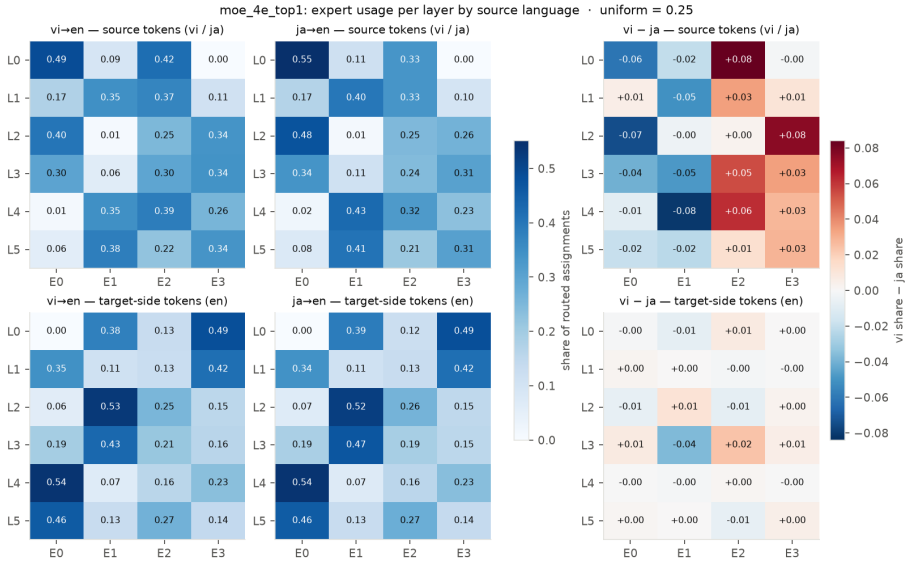


Figure 2. Expert usage per layer for `moe_4e_top1`, split by source language and by token side (top row: source tokens; bottom row: English target-side tokens). The right column shows the vi share minus the ja share. Heatmaps for the other three MoEs are in `src/part1/assets/`.

Load is well balanced; no expert collapsed. Averaged over layers, the busiest expert gets 1.05–1.09× an even share (worst single layer 1.24×), normalised load entropy is ≥ 0.998, and the balancing loss stays at 1.00–1.02. Layer 0, whose routing depends almost only on the token embedding, is the least even layer in every model; the shared-expert variant is the most balanced.

Experts specialise on source versus English far more than on vi versus ja (Fig. 2). In `moe_4e_top1` layer 0, E0 takes 49–55% of source tokens but 0% of English tokens, while E3 takes 49% of English and 0% of source. In layer 2, E1 takes 52–53% of English but 1% of source. For decoder-only translation, the useful split of work is between *reading* the input and *writing* English.

Language specialisation is weak, source-side only, and strongest in early layers. On target-side tokens the vi–ja difference is at most 0.07 (usually < 0.02): once the model writes English, routing barely depends on the source language. The clearest source-side case is `moe_shared1_3e_top1` layer 0, where E0 takes 59% of Vietnamese tokens against 32% of Japanese. The vi–ja Jensen–Shannon divergence is at most 0.056 bits, and under 0.01 bits from layer 3 on (per-layer plot: `src/part1/assets/routing_summary.png`). No expert is a "Japanese expert" or a "Vietnamese expert".

Experts specialise in token types and roles. Routing is almost deterministic per token: many frequent tokens (≥ 200 occurrences) go to the same expert 100% of the time. Table 3 shows the most consistently routed tokens in `moe_4e_top1`:

Table 3. Most consistently routed tokens in `moe_4e_top1`.

layer / expert	most consistently routed tokens	interpretation
L0 E0	そして, との, まで, 「, _私は, _mà	Japanese function words, particles and quotes
L0 E1	_company, _products, _offer, _delivery	English content words
L0 E2	_địa, _phụ, _quả, _thay, _tiền	Vietnamese syllables
L0 E3	_after, _through, _between, <2en>	English prepositions and the "write English" tag
L5 E1	<bos>, <vi>, _tham, _đăng	sequence start and language tag
L5 E2	しました。, します。, <2en>, _Number:	sentence-final verb endings and boundary tokens

Layer 0 groups tokens by script and part of speech. These clusters do not add up to a language split: E2 owns the common Vietnamese syllables, yet also takes 33% of Japanese source tokens. Some clusters are cross-lingual: in `moe_4e_top2`, layer-0 E0 takes 多くの (many), 新しい (new), _business, _system and _chuyên. The last layer groups tokens by structural role: one expert in every variant takes <bos> <vi> <ja>, and another takes sentence-final and format tokens. **At this scale, experts divide up token types and roles, not languages.**

2. Optimizers

2.1 Setup

Data. We pretrain the Q1 dense model (35.27M parameters, unchanged) from scratch on `browndw/human-ai-parallel-corpus` (HAP-E) for next-token prediction. HAP-E has 8,290 documents in 6 genres, each in 8 variants: the human `chunk_1` prompt, the human continuation `chunk_2`, and 6 LLM continuations of `chunk_1`. We split *by document* (7,462 / 414 / 414, stratified by genre), so near-duplicate continuations of a test prompt cannot leak into training. A new 32k English BPE is trained on the train split. The training stream has 37.01M tokens (25% human-written) in 512-token windows, so **1× the dataset is 2,259 steps of 16,384 tokens, or 0.0525× Chinchilla** (20 × 35.27M = 705M tokens).

Protocol (identical for every optimizer): the same initialisation and batch order; 5% linear warmup, then cosine decay to 10% of the peak LR; clipping at 1.0; bf16 autocast with fp32 states; decoupled weight decay on ≥ 2-D tensors; evaluation at every 0.1× of the dataset. **Metrics:** validation loss is the mean next-token cross-entropy over the whole validation stream (2.05M tokens). **Test BLEU** uses the parallel corpus: for each of the 414 test documents, the model writes a greedy 64-token continuation of the last 128 tokens of `chunk_1`. All 7 continuations of that prompt (human + 6 LLM, cut to 64 tokens) are references. We report corpus sacreBLEU against all 7, and against the human continuation alone.

2.2 Optimizers (all from scratch, in src/part2/optimizers.py)

We implement one optimizer from each of the first four categories of Table 1 in Wen et al. (2025), using only the `torch.optim.Optimizer` base class. g is the gradient, η the scheduled LR and λ the decoupled weight decay.

- **AdamW** (baseline): β = (0.9, 0.95), ε = 1e-8, λ = 0.1. A self-test checks it step-for-step against `torch.optim.AdamW`.
- **MARS** (variance-reduced): $c = g_t + \gamma \cdot \beta_1/(1-\beta_1) \cdot (g_t - g_{t-1})$, clipped to $\|c\| \le 1$, then an AdamW step on c (γ = 0.025, β = (0.95, 0.99)). We use the practical *approximate* variant, which reuses the previous batch's gradient instead of a second backward pass. It applies to matrices only (1-D gains use plain AdamW) and stores m, v and g_{t-1}.
- **Lion** (memory-efficient): $w \leftarrow w - \eta \cdot \mathrm{sign}(\beta_1 m + (1-\beta_1) g) - \eta\lambda w$, then $m \leftarrow \beta_2 m + (1-\beta_2) g$, with β = (0.9, 0.99) and λ = 1.0. It keeps one buffer, and every coordinate moves by exactly ±η.
- **Muon** (matrix-based): Nesterov momentum (μ = 0.95), then 5 quintic Newton–Schulz steps in bf16 that map the update $d \approx USV^T$ to about UV^T, scaled by $\sqrt{\max(1, \text{rows/cols})}$, with λ = 0.1. The fused qkv matrix is orthogonalised as three 512×512 blocks. The embedding and the RMSNorm gains take an internal AdamW step (LR 1.2e-3).
- **Self-test.** Lion and MARS match direct per-step implementations (atol 1e-5), Newton–Schulz singular values land in [0.67, 1.15], and every optimizer trains a toy transformer through the same `build_optimizer` used in training.

2.3 Learning-rate search (low-compute probe)

Each candidate LR was trained for **500 steps (0.22× the dataset)**, with the schedule compressed into those steps, and scored by its final val loss. When the best LR fell at the edge of the grid, one more point was added; all optima are interior. Other hyperparameters use literature defaults.

Table 4. Validation loss after the 500-step LR probe. Bold marks the LR chosen for the final run.

optimizer	LRs probed → val loss after 500 steps	chosen
AdamW	3e-4: 5.370 · **6e-4: 5.175** · 1.2e-3: 5.240 · 2.4e-3: 5.394	6e-4
MARS	6e-4: 5.269 · **1.2e-3: 5.140** · 2.4e-3: 5.387	1.2e-3
Lion	1.2e-4: 5.446 · **2.4e-4: 5.420** · 4.8e-4: 5.800	2.4e-4
Muon	1e-2: 4.686 · 2e-2: 4.649 · **4e-2: 4.606** · 8e-2: 4.752	4e-2

AdamW's optimum is the same as in Q1 (6e-4) on a different task. MARS takes a 2× larger LR, since its clipping and larger β_1 damp the update. Lion is the most LR-sensitive: doubling its LR costs +0.38 nats, because ±η steps cannot be absorbed by adaptive scaling. Muon is the least sensitive: an 8× LR range

moves the loss by only 0.15 nats, since orthogonalisation fixes the update's spectral scale, and all four of its probes beat AdamW's best.

2.4 Results at every 0.1× of the dataset

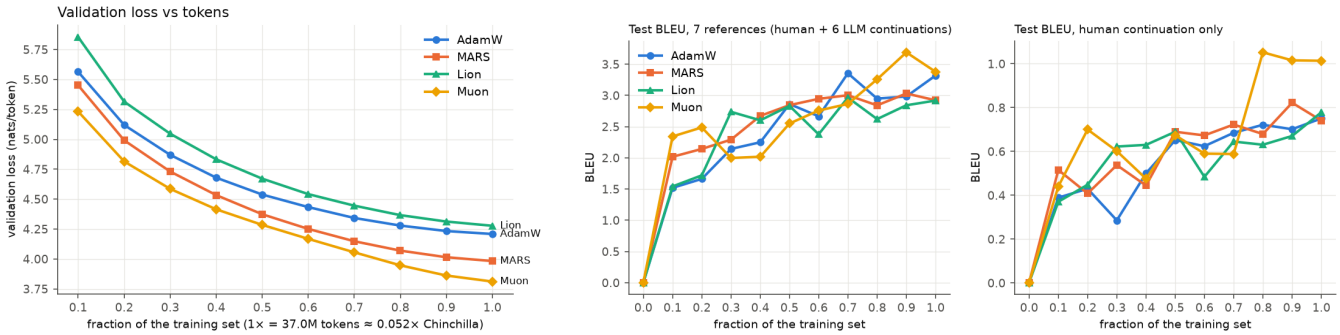


Figure 3. Validation loss (left) and test BLEU (middle: 7 references; right: human reference only) at every 0.1× of the dataset.

Table 5. Validation loss and test BLEU (7 references) at every 0.1× of the dataset (1× = 37.0M tokens). At 0.0×, every run has loss 10.456 and BLEU 0.

fraction	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
loss AdamW	5.564	5.119	4.869	4.679	4.538	4.434	4.343	4.279	4.233	4.209
loss MARS	5.451	4.991	4.730	4.533	4.373	4.251	4.148	4.070	4.015	3.982
loss Lion	5.855	5.314	5.046	4.832	4.671	4.542	4.446	4.366	4.313	4.277
loss Muon	**5.232**	**4.815**	**4.585**	**4.415**	**4.285**	**4.168**	**4.056**	**3.947**	**3.862**	**3.811**
BLEU AdamW	1.52	1.66	2.14	2.25	**2.86**	2.67	**3.35**	2.95	2.98	3.32
BLEU MARS	2.02	2.14	2.29	**2.67**	2.84	**2.95**	3.00	2.84	3.03	2.92
BLEU Lion	1.54	1.72	**2.74**	2.60	2.83	2.38	2.97	2.62	2.84	2.92
BLEU Muon	**2.34**	**2.49**	2.00	2.02	2.55	2.76	2.86	**3.26**	**3.69**	**3.38**

Table 6. Final results at 1× the dataset. *Speedup* is 37.0M divided by the tokens needed to reach AdamW's final val loss, interpolated between evaluations. ms/step is measured on one L40S; state is the total size of the optimizer's state tensors.

optimizer	LR	val loss	test PPL	BLEU 7-ref	BLEU human	speedup	ms/step	state
AdamW	6e-4	4.209	66.9	3.32	0.75	1.00×	99	269 MiB (2×)
MARS	1.2e-3	3.982	53.4	2.92	0.74	1.56×	104	404 MiB (3×)
Lion	2.4e-4	4.277	71.7	2.92	0.78	< 1×	100	**134 MiB (1×)**
Muon	4e-2	**3.811**	**44.8**	**3.38**	**1.01**	**1.77×**	108	197 MiB (1.5×)

2.5 Analysis

Training dynamics. The ranking **Muon < MARS < AdamW < Lion** holds at every evaluation from 0.1× to 1× (Fig. 3, Table 5). It also holds in the probes, on validation and test, and in every genre and in both the human and LLM subsets. With one seed per optimizer, this consistency is our main evidence that the gaps are real. Train and validation losses agree closely, so there is no overfitting.

- **Muon** is 0.33 nats below AdamW at 0.1× and 0.40 below at 1×, and reaches AdamW's final loss with 57% of the tokens (**1.77× speedup**). A plausible reason: early weight gradients are nearly low-rank, and orthogonalisation spreads each step over all singular directions, whereas Adam's per-coordinate scaling does not.
- **MARS** has a gap to AdamW that *grows* over training (0.11 → 0.23 nats; 1.56× speedup). This fits variance reduction mattering more as the LR decays and noise dominates the update.
- **Lion** starts 0.29 nats behind AdamW but ends only 0.07 behind: its fixed ±η step makes it conservative early on. Its advantage is memory, at half of AdamW's state.

Comparison with Wen et al. (2025). The qualitative picture matches the paper: matrix-based methods give the largest gains at small scale, variance-reduced AdamW gives smaller real gains, and Lion does not beat tuned AdamW. Our speedups exceed the paper's (at most about 1.4× over well-tuned AdamW) for four reasons. (1) Only AdamW's LR was tuned, and the paper shows an under-tuned baseline inflates speedups. (2) We train at 0.05× Chinchilla against the paper's 1–8×, and the paper reports

Muon's lead shrinking with more data. (3) Our intermediate points come from one cosine run, not from separately annealed runs per budget. (4) We use one seed and a 2× LR grid.

BLEU rises from 0 to about 3 for every optimizer, tracking the loss. Successive evaluations jump by ±0.5, as much as the gaps between optimizers, so BLEU cannot rank them before about 0.8×. From 0.8× on, Muon has the best human-reference BLEU (1.01–1.05 against ≤ 0.82), but MARS's clear loss advantage does not show in BLEU. The 7-reference BLEU is 3–4× the human-only value, which is the payoff of the parallel references. All models write fluent, in-register openings and then fall into greedy loops ("the literature on the literature on ..."), which caps BLEU. At this scale, validation loss is the reliable signal.

Human versus LLM text. LLM continuations are about 0.9 nats/token easier to predict than human text for every optimizer (Muon: 3.57 vs 4.53). This fits HAP-E's premise that LLM text is more homogeneous; the stream is also 75% LLM text. Muon's gain over AdamW is larger on LLM text (−0.42) than on human text (−0.33). Spoken is the easiest genre and news the hardest, for every optimizer. **Cost:** per-step overheads are MARS +5%, Lion +1% and Muon +9%, so the wall-clock speedups are about **1.62× (Muon) and 1.48× (MARS)**. Muon's state is 1.5× the parameters only because the embedding uses Adam. **Limitations:** one seed, an LR-only search on a compressed schedule, and one model size in a very-low-data regime.

3. Decoding Strategies

3.1 Setup and metrics

We use the pretrained `EleutherAI/pythia-160m` checkpoint unchanged (fp32), so there is no new checkpoint. We evaluate on exactly 1000 stories sampled at random (seed 0) from the **validation** split of `hamishivi/ROCStories`. Each story's first sentence is the *prompt* and its other four sentences are the *reference*. Every configuration decodes at most T = 70 new tokens (references: median 42 tokens, maximum 65).

Prompt format. A base LM treats a lone sentence like the end of a web paragraph: with the bare prompt, 83% of greedy outputs open with a blank line and drift into dialogue. We therefore prepend three fixed *train*-split stories (146 tokens) to every prompt. On a 48-prompt pilot, any number of shots raised greedy accuracy from 1.2% to 4.2–5.0%; beam search copied example content with one shot and a whole story with five, and three shots avoided both. Greedy with the bare prompt is kept as a **0-shot** baseline.

Metrics.

- **Perplexity (self-scored).** The decoder records $\log p_\theta(y_t \mid x, y_{<t})$ for each generated token under the *unmodified* softmax, before temperature or truncation. Strategies therefore change which token is picked, not how it is scored. Per sample, $\mathrm{PPL} = \exp(-(1/n)\,\Sigma_t \log p_\theta(y_t \mid x, y_{<t}))$. We report its mean and median, and the token-weighted corpus value.
- **Accuracy (positional match).** For reference $r_{1:m}$ and generation $y_{1:n}$, $\mathrm{Acc} = (1/m)\,\Sigma_{i \le m}\, 1[i \le n \wedge y_i = r_i]$. Unreached positions count as wrong.
- **Diagnostics.** Acc@1 is first-token accuracy; NL is the share of outputs starting with a newline; Rep-4 is the share of repeated 4-grams within a generation; D-2 is distinct-2 over all generations.

Implementation (`src/part3/decoding.py`, from scratch on `model.forward()`). We use left-padded batches with mask-derived position ids, one prefill, then one-token steps on the KV cache.

- **Greedy, top-k and top-p** share one loop and differ only in the selection rule: arg max; sampling from the renormalised k largest of logits/T; or sampling from the smallest sorted prefix of logits/T whose mass reaches p (the top token is always kept). Sampling is seeded.
- **Beam search** keeps w hypotheses per prompt as N·w rows of one batch. Each step reduces the w|V| scores s(h) + log p(v | h) to the top 2w (which guarantees w non-EOS continuations), moves EOS hypotheses to a finished pool scored by Σ log p / length, keeps the best w live ones, and reorders the KV cache by parent index. A prompt stops once w finished hypotheses beat the best live one.

- **Verification** (`sanity.py`). Beam w = 1, top-k with k = 1 and top-p with p = 10^{-6} all reproduce greedy exactly. Batched and unbatched decoding agree, and the cached log-probabilities match an uncached forward pass (mean absolute error 10^{-4}). In the full run, greedy and beam w = 1 agree on 997 of 1000 prompts; the other three are near-ties (< $3{\cdot}10^{-4}$ nats) flipped by different batch padding.

3.2 Results

Table 7. Results on 1000 ROCStories validation prompts (3-shot unless noted; 70 new tokens). All columns except PPL are percentages. Bold marks the best value per column, excluding 0-shot.

strategy	PPL mean ↓	PPL median ↓	PPL corpus ↓	Acc ↑	Acc@1	NL	Rep-4 ↓	D-2 ↑
Greedy (0-shot)	2.11	1.97	2.06	1.53	9.2	82.7	61.2	6.3
Greedy	1.72	1.67	1.70	**4.26**	**52.9**	0.4	82.6	3.7
Beam, w = 1	1.72	1.67	1.70	**4.26**	**52.9**	0.4	82.5	3.7
Beam, w = 2	1.66	1.62	1.65	3.84	47.6	7.1	81.6	4.3
Beam, w = 4	**1.58**	**1.55**	**1.57**	3.36	39.6	19.0	81.4	4.6
Top-k, k = 5	5.37	5.39	5.26	2.92	36.0	5.9	7.7	19.4
Top-k, k = 20	9.18	9.06	8.99	2.43	28.6	5.3	1.3	30.3
Top-k, k = 50	12.31	12.04	11.98	2.29	25.8	5.6	0.6	36.5
Top-k, k = 50, T = 0.7	6.30	6.30	6.12	2.90	38.3	3.9	6.6	24.1
Top-p, p = 0.5	4.85	4.89	4.64	2.99	40.3	5.3	13.6	18.2
Top-p, p = 0.9	16.69	15.81	15.81	2.23	26.7	5.7	0.5	43.0
Top-p, p = 0.95	23.21	21.12	21.35	2.11	25.6	5.4	**0.4**	**49.3**
Top-p, p = 0.9, T = 0.7	5.09	5.12	4.87	3.12	41.1	4.6	12.8	20.3

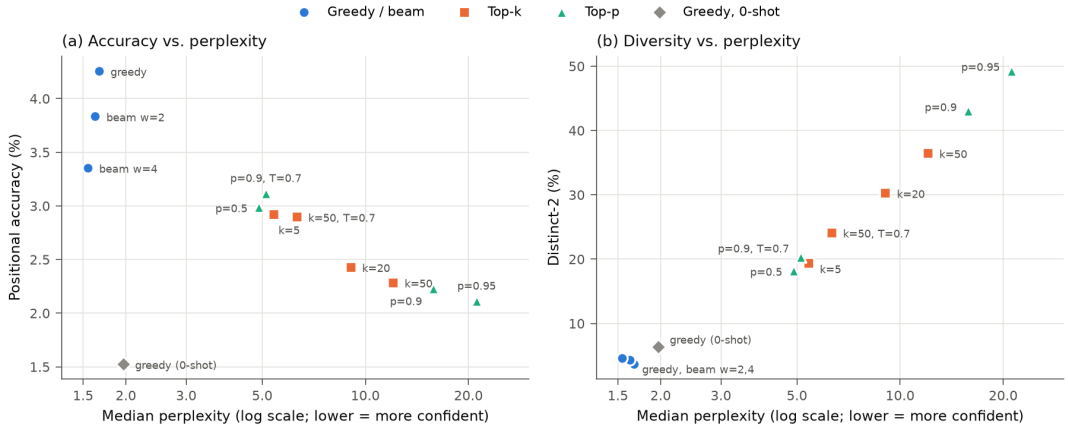


Figure 4. Each configuration placed by median perplexity (log scale) against (a) positional accuracy and (b) distinct-2. Beam w = 1 coincides with greedy and is omitted.

Perplexity tracks the amount of randomness. All three PPL columns rank the configurations identically, by how much probability mass each strategy explores: beam w = 4 (median 1.55) < beam w = 2 < greedy (1.67) ≪ top-p 0.5 (4.89) < top-k 5 < top-k 20 < top-k 50 < top-p 0.9 < top-p 0.95 (21.1). Beam search maximises sequence likelihood: the mean log-probability of a generation is −37.2 for greedy, −34.9 for w = 2 and −31.8 for w = 4. Temperature works on the same axis: T = 0.7 moves top-k 50 from 12.0 to 6.3, and top-p 0.9 from 15.8 to 5.1.

Accuracy: greedy is best, and beam search is not. Among sampling methods, accuracy mirrors perplexity (Fig. 4a), falling from 3.1% to 2.1%. Greedy is best overall at 4.26%. Beam search gets *worse* with width (3.84%, then 3.36%) despite its lower perplexity: the sequence the model finds most likely is not the human one. Wider beams increasingly open with a blank line and a dialogue loop (NL 0.4% → 7.1% → 19.0%; e.g. `\n\n"Lucy?"\n\n"Lucy?"`…). Such a loop has high *total* probability, which beam search optimises, while greedy rarely takes the less likely first step into it. A wrong first token misaligns every later position, so the Acc@1 drop (52.9% → 39.6%) carries through to Acc. Beam search is also only a heuristic: at w = 4 it beats greedy's sequence probability on just 69.9% of prompts. The metric is harsh: about half of the first tokens match (typically " He" / " She"), but from the fourth position on every strategy matches only 1–4% of tokens, since any rephrasing shifts the rest. Differences between strategies therefore come mainly from the opening tokens.

Quality versus diversity. The likelihood-maximising strategies degenerate: 83% of greedy's 4-grams repeat (81% for beam) and distinct-2 is only 3.7–4.6%. Sampling nearly eliminates repetition (0.4–13.6%) and raises distinct-2 to 18–49% (Fig. 4b). Moderate settings are the sweet spot: top-p 0.5, or top-p 0.9 / top-k 50 at T = 0.7, give the best sampling accuracy (2.9–3.1%) with only 7–14% repetition. The prompt format matters more than any decoder (3-shot greedy 4.26% against 0-shot 1.53%). Copying from the in-context examples is rare: a 6-word overlap occurs in 2.2% of beam w = 4 outputs and in none for greedy or top-p 0.9. At most 0.5% of generations end with end-of-text.

Analysis of the generations (Table 8). Greedy writes one plausible sentence and then loops. Beam search is often more on-topic but loops just as hard. Low-temperature or small-nucleus sampling gives the most story-like text: it stays with the characters but adds unmotivated events. Top-k 50 and top-p ≥ 0.9 at T = 1 are locally fluent but lose the thread within a sentence or two. No strategy recovers the reference plot; a 160M model continues these prompts with generic events.

Table 8. Example generations for the prompt *"Tom was laid off from work."* (first ~30 of 70 tokens).

	generation
Reference	It was the early afternoon and he was bored. He decided to drink some beer. Before he knew it he was drunk. The next day he decided to stop drinking.
Greedy	He was a good worker. He had a good job. He had a good family. He had a good job. He had a good job. He had a good job. ...
Beam, w = 4	He had been laid off from his job. He had been laid off from his job. He had been laid off from his job. ...
Top-k 50, T = 0.7	His family had been hurt by the loss of his parents. I asked him what he was doing. He said he had been living with his parents for a couple of months. ...
Top-p 0.9	He worked the same day as I am doing. We have not met in a while but we do not know that. He is only sick two days after going out with her friend. ...

3.3 Beam search: time versus width

Measurement. All widths decode the same 1000 prompts in 63 batches of 16 prompts (16w sequences per batch), back to back on one L40S. A warm-up batch is excluded and timing uses `torch.cuda.synchronize()`. No beam finishes early, so every width runs 63 × 70 steps. A plot with least-squares fits is in `src/part3/assets/fig_beam_cost.pdf`.

Table 9. Beam-search time and memory by width on the same 1000 prompts.

width w	total time (s)	relative	ms / step	peak GPU memory (MiB)
1	26.3	1.00×	5.96	950
2	32.6	1.24×	7.38	1272
4	47.7	1.82×	10.82	1911

Fits. Time ≈ 18.7 + 7.2w seconds: 4.2 ms of width-independent cost per step, plus 1.6 ms per beam per step. Peak memory ≈ 630 + 320w MiB. The intercept is the fp32 weights (162M parameters ≈ 620 MiB), and the slope of about 20 MiB per sequence matches one sequence's KV cache (2·12·768·~226 positions·4 B ≈ 16 MiB) plus activations.

Complexity. Let L be the number of layers, d the hidden size, |V| the vocabulary size, P the prompt length and T the generated length. With a KV cache, step t costs $O(wL(d^2 + (P+t)d))$ for the layers, $O(wd|V|)$ for the output projection and softmax, $O(w|V|)$ for the top-2w selection, and $O(wL(P+t)d)$ for reordering the cache. Hence:

$$Time(w) = O(\, w \cdot T \cdot [\, L(d^2 + (P+T)d) + d|V| \,]\,), \; Memory_{KV}(w) = O(\, w \cdot L \cdot (P+T) \cdot d\,)$$

Both are **linear in the beam width w**, with greedy as the case w = 1. Without a cache, time would be $O(wT(P+T)Ld^2)$: still linear in w, but quadratic in length.

Interpretation. The measurements follow the linear law, but with a small slope: 4× the beams cost only 1.82× the time. For a 160M model, each step is dominated by width-independent overhead (Python control flow, kernel launches, CPU–GPU syncs), and the extra rows run in parallel on an under-used GPU. On larger models, or with beams wide enough to saturate the GPU, the cost approaches proportionality in w. The beam bookkeeping itself is cheap: at equal rows in flight (64), greedy takes 8.9 ms per step against 10.8 ms for beam w = 4.